

Odoo 18 Training: Lembar Soal & Kunci Jawaban

Session 5: Advanced Logic — Overriding Methods, Decorators (@api), Domains, & ORM Queries

Modul Referensi: `jpa_sale_commission` | Standar Odoo 18.0 Community / Enterprise

SOAL 01 Dynamic Domain Filter Berbasis Many2many

Skenario Kebutuhan Bisnis:

Perusahaan ingin user dapat memfilter pilihan customer berdasarkan tag/kategori tertentu.

1. Buat field Many2many `partner_tag_ids` ke model `res.partner.category`.
2. Buat field Many2one `partner_custom_id` ke model `res.partner`.
3. Terapkan domain dinamis pada `partner_custom_id` agar opsi yang muncul hanya menampilkan partner yang memiliki tag Many2many yang dipilih di `partner_tag_ids`.

Kunci Jawaban:

Python (models/sale.py):

```
partner_tag_ids = fields.Many2many(
    'res.partner.category',
    string='Filter Partner Tags (M2M)'
)

partner_custom_id = fields.Many2one(
    'res.partner',
    string='Partner Custom (Filtered by M2M)',
    domain=[('category_id', 'in', partner_tag_ids)]
)
```

XML View (views/sale_order_views.xml):

```
<group string="Latihan Domain based on Many2many">
  <field name="partner_tag_ids" widget="many2many_tags" placeholder="Pilih tags customer..." />
  <field name="partner_custom_id" placeholder="Hanya muncul partner dengan tag di atas..." />
</group>
```

Penjelasan: Pada model `res.partner`, field tag bawaan bernama `category_id` (Many2many). Operator `'in'` mengevaluasi relasi Many2many di sisi klien sehingga pilihan otomatis terfilter sesuai tag yang dipilih.

SOAL 02 Batch Create Override (@api.model_create_multi)

Skenario Kebutuhan Bisnis:

Lakukan override method `create()` pada model `sale.order` sesuai standar modern Odoo 17 & 18. Jika user membuat dokumen tanpa menentukan nilai `commission_percent`, berikan nilai default komisi otomatis sebesar 2.5% sebelum data disimpan ke database.

Kunci Jawaban:

```
@api.model_create_multi
def create(self, vals_list):
    for vals in vals_list:
        # Injeksi nilai default jika komisi belum diisi
        if not vals.get('commission_percent'):
            vals['commission_percent'] = 2.5

    # Wajib memanggil super() dengan parameter vals_list
    return super().create(vals_list)
```

Penjelasan (Slide 25): Decorator `@api.model_create_multi` mewajibkan parameter `vals_list` (list of dicts). Pola ini memungkinkan Odoo melakukan eksekusi *batch insert* satu query SQL ke database.

SOAL 03 Write Override & Proteksi Record Terkunci (Locking)

Skenario Kebutuhan Bisnis:

Lakukan override method `write()` pada `sale.order`. Terapkan validasi bisnis: jika status Sale Order sudah `sale` (Confirmed), user dilarang mengubah nilai komisi (`commission_percent`). Jika dilanggar, gagalkan update dengan melempar `UserError`.

Kunci Jawaban:

```
def write(self, vals):
    for rec in self:
        # Jika SO sudah Confirmed dan user mencoba mengubah komisi
        if rec.state == 'sale' and 'commission_percent' in vals:
            raise UserError(_("Perubahan Ditolak! Komisi tidak boleh diubah pada Sale Order '%s' yang sudah Confirmed!",
                               rec.name))

    return super().write(vals)
```

Penjelasan (Slide 3 & 4): Parameter `vals` berisi dictionary pasangan field dan nilai baru yang sedang diubah. Kita meng-intercept proses update sebelum memanggil

SOAL 04 Unlink Override & Proteksi Penghapusan Data

Skenario Kebutuhan Bisnis:

Lakukan override method unlink(). Cegah penghapusan dokumen transaksi jika status Sale Order bukan draft (Quotation) dan bukan cancel. Tampilkan pesan UserError yang informatif.

Kunci Jawaban:

```
def unlink(self):
    for rec in self:
        if rec.state not in ('draft', 'cancel'):
            raise UserError(_("Hanya Sale Order dengan status 'Quotation (Draft)' atau 'Cancelled' yang boleh dihapus!"))
        "Order '%s' saat ini berstatus '%s'." % (rec.name, rec.state)
    return super().unlink()
```

Penjelasan (Slide 3): Method unlink() dipanggil saat user menekan Action → Delete. Validasi ini melindungi data transaksi aktif dari penghapusan yang tidak disengaja.

SOAL 05 Copy Override untuk Kustomisasi Duplikasi Dokumen

Skenario Kebutuhan Bisnis:

Ketika user menduplikasi dokumen Sale Order via tombol Action → Duplicate, buat agar nilai commission_percent pada record baru hasil duplikasi di-reset kembali menjadi 0.0%.

Kunci Jawaban:

```
def copy(self, default=None):
    self.ensure_one()
    default = dict(default or {})
    # Reset persentase komisi menjadi 0 pada dokumen baru
    default['commission_percent'] = 0.0
    return super().copy(default)
```

Penjelasan (Slide 26): Dictionary default berisi nilai-nilai yang akan menempa field bawaan record asli saat proses kloning/duplikasi dijalankan.

SOAL 06 @api.onchange dengan Dialog Warning Interaktif

Skenario Kebutuhan Bisnis:

Buat method @api.onchange pada field partner_id:

1. Jika customer berasal dari Indonesia (kode negara 'ID'), set komisi otomatis ke 10.0%.
2. Jika customer tidak memiliki email di database, munculkan popup modal dialog peringatan (*warning*) di browser.

Kunci Jawaban:

```
@api.onchange('partner_id')
def _onchange_partner_id_custom(self):
    if self.partner_id:
        # Default komisi 10% jika negara ID
        if self.partner_id.country_id.code == 'ID':
            self.commission_percent = 10.0

    # Popup warning jika email kosong
    if not self.partner_id.email:
        return {
            'warning': {
                'title': _("Missing Email"),
                'message': _("Customer '%s' tidak memiliki alamat email yang tercatat!", self.partner_id.name)
            }
        }
```

Penjelasan (Slide 22): @api.onchange beroperasi di memori form view browser sebelum data di-save. Mengembalikan dictionary dengan key 'warning' memicu Odoo menampilkan pop-up alert interaktif.

SOAL 07 @api.constrains untuk Integritas Server-Side

Skenario Kebutuhan Bisnis:

Buat aturan validasi bisnis pada field commission_percent:

1. Komisi tidak boleh melebihi 50.0%.
2. Komisi tidak boleh bernilai negatif (< 0.0%).
3. Jika dilanggar saat tombol Save ditekan, batalkan transaksi dan lempar ValidationError.

Kunci Jawaban:

```
@api.constrains('commission_percent')
def _check_commission_percent(self):
    for rec in self:
        if rec.commission_percent > 50.0:
            raise ValidationError(_(
                "Komisi tidak boleh melebihi 50%%! (Nilai saat ini: %.2f%%)",
                rec.commission_percent
            ))
        if rec.commission_percent < 0.0:
            raise ValidationError(_("Komisi tidak boleh bernilai negatif!"))
```

Penjelasan (Slide 24): Berbeda dengan onchange, @api.constrains dieksekusi di server saat operasi create() atau write(). Aturan ini menjaga integritas database dan tidak dapat di-bypass oleh API eksternal atau import CSV.

SOAL 08 Domain Polish Notation di Search View XML

Skenario Kebutuhan Bisnis:

Pada search view sale.order, tambahkan 3 buah filter baru:

1. Filter **"Draft or Confirmed"** dengan operator Explicit OR ('|').
2. Filter **"Not Cancelled"** dengan operator Negation NOT ('!').
3. Filter **"High Commission (> 10%)"** dengan aturan XML escaping yang benar.

Kunci Jawaban:

```
<record id="view_sales_order_filter" model="ir.ui.view">
  <field name="name">view_sales_order_filter</field>
  <field name="model">sale.order</field>
  <field name="inherit_id" ref="sale.view_sales_order_filter"/>
  <field name="arch" type="xml">
    <xpath expr="//filter[@name=my_sale_orders_filter]" position="after">

      <!-- 1. Explicit OR Operator ('|') (Slide 12) -->
      <filter string="Draft or Confirmed (OR '|')">
        domain="[('state', '=', 'draft'), ('state', '=', 'sale')]"
        name="filter_draft_or_sale"/>

      <!-- 2. Negation NOT Operator ('!') (Slide 12) -->
      <filter string="Not Cancelled (NOT '!')">
        domain="[('state', '=', 'cancel')]"
        name="filter_not_cancel"/>

      <!-- 3. Comparison dengan XML Escaped &gt; (Slide 14 & 17) -->
      <filter string="High Commission (> 10%)">
        domain="[('commission_percent', '>', 10)]"
        name="filter_high_comm"/>

    </xpath>
  </field>
</record>
```

Penjelasan (Slide 10, 12, 17): Domain Odoo dievaluasi dari kiri ke kanan (Prefix Polish Notation). Di file XML, karakter pembanding > dan < wajib di-escape menjadi > dan < untuk tidak merusak parser XML.

SOAL 09 ORM Search, Mapped & Agregasi Data Relasi (Hitung Komisi Loyalitas)**Skenario Kebutuhan Bisnis:**

Buatlah sebuah method tombol pada sale.order bernama action_apply_loyalty_commission() yang mendemonstrasikan unsur query ORM Odoo:

1. Cari seluruh order sebelumnya yang sudah berstatus 'sale' (Confirmed) milik customer tersebut menggunakan `self.env['sale.order'].search(domain)`.
2. Hitung total nilai belanja historis menggunakan method recordset `.mapped('amount_total')` dan `sum()`.
3. Jika total belanja historis $\geq$ Rp 10.000.000 atau memiliki ≥ 3 order confirmed, naikan `commission_percent` menjadi 15.0%. Jika tidak, set ke 5.0%.
4. Catat pesan pemberitahuan ke chatter menggunakan `self.message_post()` dan munculkan notifikasi popup ke layar.

Kunci Jawaban:

```
def action_apply_loyalty_commission(self):
    self.ensure_one()
    if not self.partner_id:
        raise UserError(_("Silakan pilih Customer terlebih dahulu sebelum menghitung komisi!"))

    # 1. ORM Search: Cari histori order confirmed milik customer ini
    domain = [
        ('partner_id', '=', self.partner_id.id),
        ('state', '=', 'sale'),
        ('id', '!=', self.id)
    ]
    past_orders = self.env['sale.order'].search(domain)

    # 2. Agregasi ORM menggunakan mapped() dan sum()
    total_spent = sum(past_orders.mapped('amount_total'))
    order_count = len(past_orders)

    # 3. Logika bisnis penentuan komisi
    if total_spent >= 10000000 or order_count >= 3:
        self.commission_percent = 15.0
        msg = _("Customer Loyal! Histori: %d order (Total: Rp %s). Komisi dinaikkan ke 15%%.",
                order_count, f"{total_spent:,.2f}")
    else:
        self.commission_percent = 5.0
        msg = _("Customer Reguler. Histori: %d order (Total: Rp %s). Komisi diset ke 5%%.",
                order_count, f"{total_spent:,.2f}")

    # 4. Catat ke chatter SO & kembalikan notifikasi
    self.message_post(body=msg)
    return {
        'type': 'ir.actions.client',
        'tag': 'display_notification',
        'params': {
            'title': _("Perhitungan Komisi Loyalitas (ORM)"),
            'message': msg,
            'sticky': False,
            'type': 'success',
        }
    }
```

Penjelasan (Slide 6 & 7): `self.env['model_name'].search(domain)` mengonversi domain Odoo menjadi query SQL `SELECT * FROM sale_order WHERE ....` Method `.mapped('field_name')` mengekstrak seluruh nilai kolom dari kumpulan recordset menjadi list Python yang bersih.

SOAL 10 Dynamic Domain Builder dengan expression.OR & expression.AND**Skenario Kebutuhan Bisnis:**

Buatlah sebuah method action_find_similar_orders() untuk mencari order-order serupa dengan membangun domain dinamis di Python (Slide 16):

1. Kondisi A: Order milik customer yang sama (partner_id).
2. Kondisi B: Order dengan nominal mendekati ($\pm 30\%$ dari total order saat ini).
3. Gabungkan Kondisi A dan B menggunakan expression.OR([dom_a, dom_b]).
4. Tambahkan syarat wajib bahwa status harus 'sale' menggunakan expression.AND([combined, [('state', '=', 'sale')]]).
5. Kembalikan action window bertipe ir.actions.act_window untuk menampilkan daftar order yang cocok ke layar user.

Kunci Jawaban:

```
def action_find_similar_orders(self):
    self.ensure_one()
    from odoo.osv import expression

    # Kondisi A: Customer yang sama
    dom_a = [('partner_id', '=', self.partner_id.id)]

    # Kondisi B: Rentang nominal yang mirip (+/- 30%)
    min_amount = self.amount_total * 0.7
    max_amount = self.amount_total * 1.3
    dom_b = [('amount_total', '>=', min_amount), ('amount_total', '<=', max_amount)]

    # Gabungkan secara dinamis: (A OR B) AND state == 'sale'
    combined_dom = expression.OR([dom_a, dom_b])
    final_domain = expression.AND([combined_dom, [('state', '=', 'sale')]])

    return {
        'name': _("Similar Orders (Dynamic Domain ORM)"),
        'type': 'ir.actions.act_window',
        'res_model': 'sale.order',
        'view_mode': 'list,form',
        'domain': final_domain,
        'target': 'current',
    }
```

Penjelasan (Slide 16): Helper odoo.osv.expression.OR dan expression.AND secara otomatis menyisipkan operator Polish Notation ('|' dan '&') dengan tanda kurung tepat, tanpa developer harus menghitung urutan prefix secara manual.

SOAL 11 Parameterized Native SQL & Re-Wrapping ke ORM (self.env.cr & browse)**Skenario Kebutuhan Bisnis:**

Jelaskan dan tuliskan kode method coba_query() untuk menjalankan Native SQL Query secara aman di Odoo:

1. Eksekusi query dengan parameter tuple %s untuk mencegah celah keamanan *SQL Injection*.
2. Ambil hasil query berupa list of tuples (fetchall()) dan single count (fetchone()).
3. Tunjukkan cara mengubah raw ID hasil SQL kembali menjadi ORM recordset menggunakan browse(ids) (Slide 8).

Kunci Jawaban:

```
def coba_query(self):
    # 1. Parameterized SQL query (%s aman dari SQL Injection)
    self.env.cr.execute(
        "SELECT id, name FROM res_partner WHERE is_company = %s LIMIT 3",
        (True,)
    )
    rows = self.env.cr.fetchall() # Mengambil list of tuples
    print("Hasil fetchall:", rows)

    # 2. Mengambil single scalar value
    self.env.cr.execute("SELECT COUNT(*) FROM sale_order WHERE state = %s", ('sale',))
    count = self.env.cr.fetchone()[0]
    print("Total Confirmed SO:", count)

    # 3. Mengubah raw SQL IDs kembali ke ORM Recordset (Slide 8)
    self.env.cr.execute("SELECT id FROM sale_order WHERE amount_total > %s LIMIT 5", (1000000,))
    raw_ids = [row[0] for row in self.env.cr.fetchall()]
    orders = self.env['sale.order'].browse(raw_ids) # Full ORM access restored!
    for o in orders:
        print("Akses field ORM:", o.name, o.partner_id.name)
```

Penjelasan (Slide 8 & 9): self.env.cr adalah active database cursor. Selalu gunakan placeholder %s dengan tuple parameter, jangan pernah menggunakan f-string (f"..."). Gunakan browse(ids) untuk mengembalikan ID mentah SQL menjadi objek recordset Odoo.